

Архитектура проекта NAV SaaS MVP

Версия 2026-06-29

Пробный вертикальный срез SaaS-платформы для автоматизации выпуска смет (ГСН-2022, ФГИС ЦС). Целевая схема: ADMIN FR / USER FR -> БК -> АС/JWT -> storage.

Общая схема

Два основных процесса: API-сервер (run.bat, порт 8080) и отдельный calc worker (run-calc-worker.bat). Браузерный UI (web/) обращается к API по REST с JWT в cookie. Сервер раздаёт статику, обслуживает auth и CRUD, читает справочник ГСН. Worker забирает задачи из очереди PostgreSQL и обогащает строки смет данными ГСН.

Клиенты: web/ (основной UI), web/admin.html (админка)
Процессы: backend/cmd/server, backend/cmd/calc_worker
Хранилища: data/app.json, PostgreSQL app_*, PostgreSQL gsn.*/fgis_cs.*

Процессы

- API + frontend (run.bat) — HTTP-сервер, auth, CRUD, GSN API, раздача web/
- Calc worker (run-calc-worker.bat) — асинхронный расчёт позиций смет из очереди

Backend (Go)

Монолитный HTTP-сервер с модульной внутренней структурой. Точка входа — backend/cmd/server/main.go.

```
backend/  
  cmd/server/      — основной API  
  cmd/calc_worker/ — воркер расчёта смет  
  cmd/import_*     — CLI-импорт справочников  
  internal/api/    — маршруты, middleware, handlers  
  internal/auth/   — JWT (HMAC SHA-256), пароли, cookie-сессии  
  internal/domain/ — доменные модели  
  internal/store/  — FileStore: гибрид JSON + PostgreSQL  
  internal/gsn/    — справочник ГСН-2022 и ФГИС  
  internal/calcworker/ — логика воркера и Manager пула goroutine  
  internal/presence/ — блокировки смет, лицензионные сессии (in-memory)
```

API-сервер (internal/api/server.go)

- http.ServeMux с префиксом /api/
- Middleware: CORS, JWT из cookie, проверки authorized / admin
- Статика из APP_WEB_DIR; админ-страница /admin

Группы эндпоинтов:

- Auth: /api/auth/login, logout, register, /api/me
- Организация: /api/companies, /api/users
- Предметная область: /api/constructions, /api/objects, /api/estimates
- Расчёт: GET /api/estimates/{id}/calc-status
- GSN: /api/gsn/hierarchy, record, regions, fgis-sets и др.
- Админ: /api/admin/estimate-locks, /api/admin/licenses
- Настройки: GET/PUT /api/settings (число calc worker-ов)

Аутентификация (internal/auth)

- Роли: super_admin, company_admin, user
- Логин по компании + ФИО или e-mail + пароль
- JWT в HTTP-only cookie; пароли SHA-256 с солью

Хранилище (internal/store/file_store.go)

Гибридная модель:

- Пользователи, компании, лицензии, настройки — data/app.json
- Стройки, объекты, сметы, строки, очередь — PostgreSQL (app_*, estimate_calc_jobs) при APP_DATABASE_URL

Целевая схема описана в db/schema.sql.

Модуль ГСН (internal/gsn)

Отдельное подключение APP_GSN_DATABASE_URL. Схема — db/gsn_schema.sql:

- gsn.supplements, gsn.hierarchy, gsn.base_info — иерархия ГСН-2022
- fgis_cs.* — наборы и строки ФГИС ЦС (цены по районам)
- Импорт — CLI import_regions, import_resource_codifier, import_fgis_cs и scripts/

Presence (internal/presence)

- EstimateLocks — эксклюзивная блокировка сметы при редактировании (TTL ~90 с)
- LicenseSessions — учёт активных лицензий по подразделам ГСН (in-memory)

Асинхронный расчёт смет

Ключевая идея: источник истины — текстовая строка (raw_text), таблица — проекция, очередь — транспорт для worker-ов.

1. UI -> PUT /api/estimates (parsed lines)
2. API -> app_estimate_lines (calc_status=queued) + estimate_calc_jobs
3. Calc worker -> ClaimEstimateCalcJob (FOR UPDATE SKIP LOCKED)
4. Worker -> gsn.GetRecordDetail(code, fgisSet, district)
5. Worker -> calc_json, calc_status=done
6. UI polling GET calc-status -> обогащение таблицы

- Очередь estimate_calc_jobs: queued | leased | done | failed | dead
- Lease ~45 с; retry с backoff; после max_attempts -> dead
- Worker не встроен в server — только calc_worker (1–16 goroutine по настройке)
- Парсинг объёма из raw_text — на backend (normalizeEstimateItem, quantity_expr.go)

Frontend (web/)

SPA без фреймворка — vanilla JS + HTML + CSS.

- index.html + app.js — основное приложение
- admin.html + admin.js — админ-панель
- shared.js, login-draft.js, styles.css, regions.json

Разделы UI:

- База -> ГСН-2022 (иерархия, поиск), позиции пользователя
- Стройки -> иерархия Стройка -> Объект -> Смета
- Настройки (число worker-ов расчёта)

Редактор сметы: текстовый режим (формат «Исходные данные») и табличный (до расчёта — шифр + объём; после worker — полная строка из calc_json). Состояние редактора — sessionStorage.

Базы данных

APP_DATABASE_URL:

app_constructions, app_construction_objects, app_estimates,
app_estimate_lines, estimate_calc_jobs, app_settings

APP_GSN_DATABASE_URL:

gsn.supplements, gsn.hierarchy, gsn.base_info, fgis_cs.*

Переменные окружения: APP_ADDR, APP_WEB_DIR, APP_DATA_PATH, APP_DATABASE_URL,
APP_GSN_DATABASE_URL, APP_JWT_SECRET.

Вспомогательные инструменты

- scripts/import_gsn_*.py — импорт ГСН в PostgreSQL
- prep_gsn_books.py — подготовка книг ГСН
- tools/ — тестовые и отладочные скрипты
- backup-sources.ps1 — резервные копии исходников

Текущее состояние и эволюция

- Сделано: PG для смет и очереди; отдельный calc worker; GSN-модуль; JWT; мультитенантность
- Частично: users/companies в JSON (app.json), схема PG есть в db/schema.sql
- Запланировано: полный переход store на PG; вынос auth; outbox/events; granular permissions

Архитектура MVP: один Go-бинарник + worker-процесс + статический frontend. Очередь — таблица PostgreSQL с FOR UPDATE SKIP LOCKED, без message broker.